

Nodejs

```
// express js => nodejs frame work

import express from "express"; // es6

const app = express();

app.use(express.json());

let user = [{ name: "FIIT", age: 24, college: "DKM" }];

app.get("/", (req, res) => {
  return res.json(user);
});

app.post("/post", (req, res) => {
  console.log(req.body);

  user.push(req.body);

  return res.json(user);
});

app.listen(3000, () => {
  console.log("Server is up and running");
});
```

Theory

1. Express.js Basics

- **Definition:** Express.js is a lightweight Node.js framework for building web applications and APIs.
- **Features:**
 - Middleware support.
 - Routing system.
 - Easy integration with databases.
 - JSON handling.

2. Middleware

- **Definition:** Functions that execute during the request-response cycle.
- **Types:**
 - Application-level (`app.use()`).
 - Router-level (`router.use()`).
 - Built-in (`express.json()`, `express.static()`).
 - Error-handling middleware.

3. Routing

- **Definition:** Defines how the server responds to client requests.
- **Methods:**
 - `app.get()` → fetch data.
 - `app.post()` → create data.
 - `app.put()` → update data.
 - `app.delete()` → delete data.
- **Route parameters:** `app.get("/user/:id", ...)`.

4. JSON Handling

- `express.json()` middleware parses incoming JSON requests.
- `res.json()` sends JSON responses.

5. REST API Principles

- **Stateless** → each request independent.
- **CRUD operations** → Create, Read, Update, Delete.
- **Resource-based URLs** → `/users`, `/products`.
- **HTTP methods** → GET, POST, PUT, DELETE.



Coding Round Questions

- Express basics → What is Express.js and why is it used?
- Middleware → Explain middleware and give examples.
- Routing → Write routes for GET, POST, PUT, DELETE.
- JSON handling → Show how to parse and send JSON data.
- REST principles → Explain REST API principles with examples.
- Error handling → How do you handle errors in Express.js?



Machine Coding Round Challenge

Problem: User Management API

Requirements:

- Create an Express.js server.
- Define routes:
 - GET /users → return all users.
 - POST /users → add a new user.
 - PUT /users/:id → update user by ID.
 - DELETE /users/:id → delete user by ID.
- Use `express.json()` middleware for parsing.
- Maintain users in an array.